‘How to’ guide of Coding in Elise Woodward’s Thesis

Table of Contents

Table of Contents

Chapters 1 and 2- Model Formulation

To run one patch simulations or two patch simulations. There are two methods of coding, one solving an integral and one running the function ode23.

Piecewise simulation method

Pachepsky2PM

This code is the outer code that will call pachepsky2Pf using ode 23. Here you may want to change the probability of migration, the initial conditions, and whether you want to plot the before impulse or after impulse dynamics.

pachepsky2Pf

This code is what ode23 reads as the ode file. Here you may want to change the model parameters.

Integral Method

int_eval_2p

This function evaluates the necessary integrals

INT_pachepskyf_after_2P

This function calculates the density after the year, at time 1+, etc

Other

implicitplot_thesis_equil

This function takes the coexistence equations and plots them as ellipses. It does so by using contours over a grid. It plots three cases and attempts to find the POI. You may want to change the parameters and the outputted file name.

Chapter 3- Benefit of Dispersal

w_1_w_2_asymp

Plots the components of the first order correction in two adjacent plots

overlay_asymp_benefit_consumer:

runs:

-*run_model_mu_consumer*

-pachepsky 2pf_mu

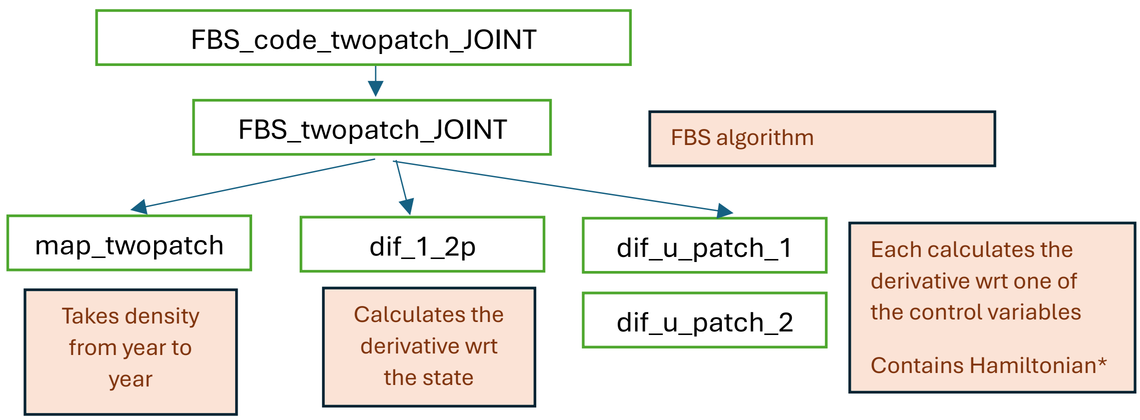
overlay_asymp_benefit_resource:
runs:

-*run_model_mu_resource*

-pachepsky 2pf_mu

Chapter 4- Optimal Control

Forward Backward Sweep for Timing



FBS_code_twopatch_JOINT
This is the script that calls the inner functions and it also calls the function that executes the FBS algorithm. It also plots the results.

Here you may want to change:

-parameter values

-initial conditions

-final time

Calls: FBS_twopatch_JOINT

FBS_twopatch_JOINT
This is the script that is called in FBS_twopatch_JOINT and it executes the FBS algorithm. It calls various functions.

Here you might change:

-tolerance

-initial guess

-terminal condition on λ (transversality condition)

-what map Φ dictates the dynamics (in my case it's often *map_twopatch*)

-how the adjoint is updated based on the adjoint equation

Calls:

map_twopatch -to determine the dynamics of the system (straightforward)

diff_1_2p – to determine the Jacobian of the dynamics (complicated)

dif_u_patch_1- determines the optimality condition

dif_u_patch_2- determines the optimality condition

diff_1_2p

Calculates the derivative of map_twopatch that will be used to solve the adjoint equation. It does not contain the Hamiltonian.

You may want to change:

-Map it takes the derivative wrt

It calls: map_twopatch

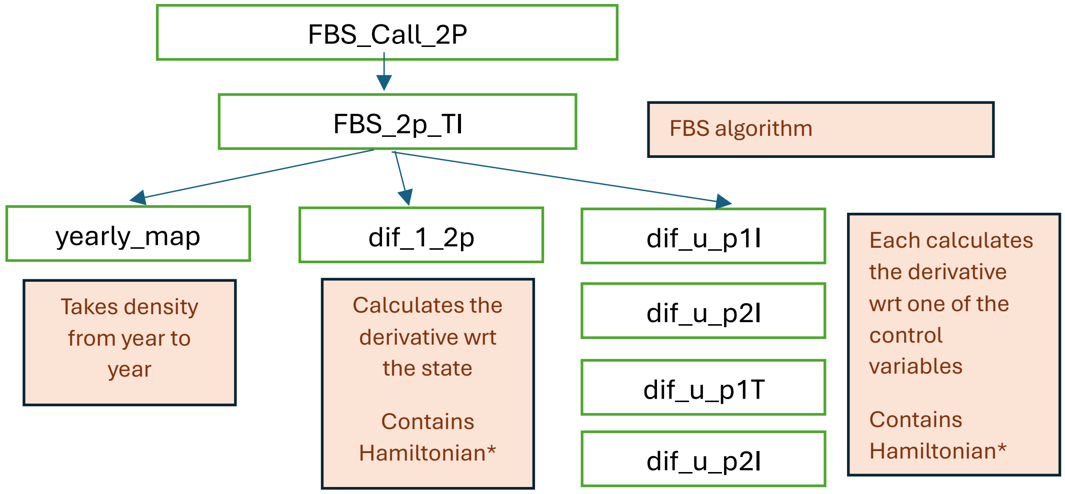
dif_u_patch1

This function calculates the derivative of the Hamiltonian wrt u.

It works like this:

1. Perturb the given u (either u_1 or u_2) by a small amount forward and backward
2. Then, run the map_twopatch function to get the components of the perturbed Hamiltonian
3. Then, calculate the perturbed Hamiltonian using one of the regular u (if you're doing dif_u_patch1- then u2 will be the regular one), and everything else will be left for H_left and right for H_right. The reason v and w multiplied are in the next time step (they are right and left) is that they are already like that in the Hamiltonian.

Forward Backward sweep for timing and intensity



FBS_Call_2P

May want to change: parameters, file name

FBS_2p_TI

May want to change:

-initial guess

- transversality condition

-adjoint equation

yearly_map

This map uses a slightly different system than the map_twopatch, because there may be different timing on each patch.

Nothing to change here